

Indicații

- Testul conține 3 subiecte, durează 120 de minute și valorează 50 de puncte, cu alte 5 bonus.
- Se pot obține punctaje parțiale pentru un subiect numai dacă acest lucru este specificat.
- Pentru a fi punctată, o rezolvare **trebuie** să includă și metoda de verificare a funcționalității acesteia.
- Pentru subiectul 1, pentru afișarea la ecran este recomandată utilizarea macroului `PRINTF32`.
- Pentru subiectul 2, pentru afișarea la ecran este obligatorie utilizarea funcției de bibliotecă `printf()`.
- Ordinea de rezolvare a subiectelor este la alegerea voastră.
- Subiectele se rezolvă pe mașina virtuală de PCLP2.

Subiecte

Subiectul 1

Tocmai ați fost încolțiți de dragonul Smaug, însă acesta are mai multe slăbiciuni: împărțirile și operațiile pe biți. Dovediți că îi sunteți superiori, înfruntându-l cu ajutorul acestora!

- [5p] Prima slăbiciune a dragonului sunt literele cărora, dacă le luăm codul ASCII și îl împărțim la 5, obținem un rest mai mic sau egal cu 1. Le vom numi litere **periculoase**. Pentru a le identifica, parcurgeți șirul `danger`, salvat în secțiunea `.data`, și afișați literele **periculoase**.
- [5p] Pentru a putea descoperi și celelalte slăbiciuni ale dragonului, va trebui să setați valoarea de retur a executabilului la 55. Puteți verifica această valoare folosind comanda `echo $?` după rularea executabilului.
- [5p] A doua slăbiciune a dragonului sunt numerele **grele** — acelea care au bitul cel mai semnificativ (MSB) setat și pentru care numărul format prin concatenarea celui de al treilea și al patrulea octet este strict mai mare decât 255 (țineți cont că avem sistemul în format little endian). Pentru testare, folosiți variabila `heavy_number` salvată în secțiunea `.data`. Dacă aceasta este grea, afișați mesajul “The number is heavy”, altfel, “The number is not heavy”. (**Hint**: Comparația cu 255 se face pe tipul de date `dword` cu restul de 2 bytes zeroizați.)
- [5p] Pentru a câștiga bătălia cu dragonul, va trebui să parcurgeți vectorul `battle`, salvat în secțiunea `.data`, și să afișați numerele **grele** în format hexazecimal.

Subiectul 2

O variantă eficientă și performantă de a reține mulțimile de perechi `<key, value>` este folosirea unui `map`.

Un `map` este o structură de date care asociază fiecărei chei (`key`) o valoare (`value`), permițând stocarea și accesarea rapidă a valorilor pe baza cheilor unice.

Astfel, pentru o cheie dată, se poate stoca și recupera valoarea asociată eficient, facilitând operații precum inserarea, căutarea și modificarea valorilor.

```
map[key] = value
```

Atenție: `map`-ul va conține doar litere mici.

- [5p] Implementați funcția `void insert_in_map(char *map, int *size, char *sir)`, care parcurge șirul `sir` salvat în secțiunea `.data`, salvează în `map`-ul `map` din secțiunea `.bss` fiecare literă împreună cu numărul său de apariții și afișează, la final, litera care apare de cele mai multe ori și numărul său de apariții.
- [5p] Implementați funcția `void print_map(char *map, int *size)`, care afișează, pentru fiecare **cheie** introdusă în `map` (de dimensiune maximă `size`) numărul său de apariții doar dacă acesta este diferit de zero. Formatul este `<Litera>: <nr_apariții>\n`. Alocați șirul “it’s not that hard.” pe stivă, apoi apelați funcția `insert_in_map` pentru acest șir. Pentru testare, apelați funcția `print_map`.
- [5p] Implementați funcția `void card_map(char *map, int *size)` care afișează cardinalul mulțimii reținute în `map`-ul `map` de dimensiune maximă `size` octeți. Prin cardinalul mulțimii ne referim la numărul de litere unice introduse în `map` (NU suma valorilor fiecărei chei). Pentru testare, folosiți `map`-ul populat la punctele anterioare.
- [5p] Implementați funcția `void expand_map(char *map, int *size)`, care parcurge întregul `map` (fiecare literă din alfabet), de dimensiune maximă `size`, și pentru fiecare cheie actualizează valoarea asociată folosind formula: $((2 * value + 1) \ll 7) \% 5$. Apelați apoi funcția `print_map`.

Subiectul 3

- a) [5p] Dacă vrei ceva făcut bine, trebuie să-l compilezi tu însuși. Faceți în așa fel încât să se afișeze mesajul “Yey, you found it!”, folosindu-vă de fișierul din `print/print.c`.
- b) [5p] Inspectați executabilul `only_one/one` și găsiți o modalitate pentru a declanșa afișarea mesajului “Nice, you got here!”.
- c) [5p] Inspectați executabilul `long_way_to_go/longway` și găsiți o modalitate pentru a declanșa afișarea mesajului “Damnnn, you’re on fire!”.

(**Hint:** Puteți afla adresa argumentului `argc` folosind `gdb` și comanda `print (int *)C`)